

interpreter prints out the rest of the text. The Python we use in this book is version 3. If you see `Python 2...` on the first line, then you have an older version of the Python interpreter. Try running `python3` in the command-line interface to see if Python version 3 is already installed on the computer. If not, either upgrade the version of Python or find a different computer that has `python3`. The differences between Python 2 and 3 can be subtle and difficult to understand for new programmers, so it's important to get the right version. There are also differences between every minor release version of Python, for example, between versions 3.8 and 3.9. Check the online documentation at <https://docs.python.org> to find the changes.

The interpreter continues prompting for Python expressions, evaluating them, and printing their values until you ask it to stop. The Python interpreter prompts for expressions using `>>>`. If you want to terminate the interpreter and you're using an IDE, you typically quit the IDE application. For interpreters launched in a command-line interface, you can press Ctrl-D or sometimes Ctrl-C to exit the Python interpreter. In this book, we show all of the Python examples being launched from a command line, with a command that starts with `$ python3`.

In [Listing 1-1](#), you can see that simple arithmetic expressions produce results like other programming languages. What might be less obvious is that small integers and very large integers (bigger than what fits in 32 or 64 bits of data) can be calculated and used just like smaller integers. For example, look at the result of the expression `10**27 + 1`. Note that these big integers are not the same as floating-point numbers. When adding integers and floating-point numbers as in `10**27 + 1.0001`, the big integer is converted to floating-point representation. Because floating-point numbers only have enough precision for a fixed number of decimal places, the result is rounded to `1e+27` or 1×10^{27} .

Whitespace syntax is important even when using the Python interpreter interactively. Nested expressions *use indentation instead of a visible character* to enclose the expressions that are evaluated conditionally. For example, Python `if` statements demarcate the `then`-expression and the `else`-expression by indentation. In C++ and JavaScript, you could write

```
if (x / 2 == 1) {do_two(x)}  
else {do_other(x)}
```

The curly braces enclose the two expressions. In Python, you would write

```
if x / 2 == 1:
    do_two(x)
else:
    do_other(x)
```

You must indent the two procedure call lines for the interpreter to recognize their relation to the line before it. You must be consistent in the indentation, using the same tabs or spaces, on each indented line for the interpreter to know the nested expressions are at the same level. Think of the indentation changes as replacements for the open curly brace and the close curly brace. When the indent increases, it's a left brace. When it decreases, it is a right brace.

When you enter the preceding expression interactively, the Python interpreter prompts for additional lines with the ellipsis prompt (...). These prompts continue until you enter an empty line to signal the end of the top-level expression. The transcript looks like this, assuming that `x` is 3 and the `do_other()` procedure prints a message:

```
>>> if x / 2 == 1:
...     do_two(x)
... else:
...     do_other(x)
...
Processing other value
>>>
```

Note, if you've only used Python 2 before, the preceding result might surprise you, and you should read the details of the differences between the two versions at <https://docs.python.org>. To get integer division in Python 3, use the double slash (`//`) operator.

Python requires that the indentation of **logical lines** be the same if they are at the same level of nesting. Logical lines are complete statements or expressions. A logical line might span multiple lines of text, such as the previous `if` statement. The next logical line to be executed after the `if` statement's `then` or `else` clause should start at the same indentation as the `if` statement does. The deeper indentation indicates statements that are to be executed later (as in a function definition), conditionally (as in an `else` clause), repeatedly (as in a loop), or as parts of larger construct (as in a class definition). If you have long expressions that you would prefer to split across multiple lines, they either

- Need to be inside parentheses or one of the other bracketed expression types (lists, tuples, sets, or dictionaries), or
- Need to terminate with the backslash character (\) in all but the last line of the expression

Inside of parentheses/brackets, the indentation can be whatever you like because the closing parenthesis/bracket determines where the expression ends. When the logical line containing the expression ends, the next logical line should be at the same level of indentation as the one just finished. The following example shows some unusual indentation to illustrate the idea:

```
>>> x = 9
>>> if (x %
...     2 == 0):
...     if (x %
...     3 == 0):
...         'Divisible by 6'
...     else:
...         'Divisible by 2'
... else:
...     if (x %
...     3 == 0):
...         'Divisible by 3'
...     else:
...         'Not divisble by 2 or 3'
...
'Divisible by 3'
```

The tests of divisibility in the example occur within parentheses and are split across lines in an arbitrary way. Because the parentheses are balanced, the Python interpreter knows where the `if` test expressions end and doesn't complain about the odd indentation. The nested `if` statements, however, must have the same indentation to be recognized as being at equal levels within the conditional tests. The `else` clauses must be at the same indentation as the corresponding `if` statement for the interpreter to recognize their relationship. If the first `else` clause is omitted as in the following example,

```
>>> if (x %
...     2 == 0):
...     if (x %
...     3 == 0):
```

```

...         'Divisible by 6'
...     else:
...         if (x %
...             3 == 0):
...             'Divisible by 3'
...         else:
...             'Not divisble by 2 or 3'
...
'Divisible by 3'

```

then the indentation makes clear that the first `else` clause now belongs to the `if (x % 2 == 0)` and not the nested `if (x % 3 == 0)`. If `x` is 4, then the statement would evaluate to `None` because the `else` clause was omitted. The mandatory indentation makes the structure clearer, and mixing in unconventional indentation makes the program very hard to read!

Whitespace inside of strings is important and is preserved. Simple strings are enclosed in single (') or double (") quote characters. They cannot span lines but may contain escaped whitespace such as newline (`\n`) or tab (`\t`) characters, e.g.,

```

>>> "Plank's constant:\n quantum of action:\t6.6e-34"
"Plank's constant:\n quantum of action:\t6.6e-34"
>>> print("Plank's constant:\n quantum of action:\t6.6e-34")
Plank's constant:
quantum of action:    6.6e-34

```

The interpreter reads the double-quoted string from the input and shows it in *printed representation* form, essentially the same as the way it would be entered in source code with the backslashes used to escape the special whitespace. If that same double-quoted string is given to the `print` function, it prints the embedded whitespace in output form. To create long strings with many embedded newlines, you can enclose the string in triple quote characters (either single or double quotes).

```

>>> """Python
... enforces readability
... using structured
... indentation.
... """
'Python\nenforces readability\nusing structured\nindentation.\n'

```

Long, multiline strings are especially useful as documentation strings in function definitions.

You can add comments to the code by starting them with the pound symbol (#) and continuing to the end of the line. Multiline comments must each have their own pound symbol on the left. For example:

```
def within(x, lo, hi):    # Check if x is within the [lo, hi] range
    return lo <= x and x <= hi # Include hi in the range
```

We've added some color highlights to the comments and reserved words used by Python like `def`, `return`, and `and`, to improve readability. We discuss the meaning of those terms shortly. Note that comments are visible in the source code files but not available in the runtime environment. The documentation strings mentioned previously are attached to objects in the code, like function definitions, and are available at runtime.

Dynamic Typing

The next most noticeable difference between Python and some other languages is that it uses **dynamic typing**. That means that the data types of variables are determined at runtime, not declared at compile time. In fact, Python doesn't require any variable declarations at all; simply assigning a value to variable identifier creates the variable. You can change the value and type in later assignments. For example,

```
>>> x = 2
>>> x
2
>>> x = 2.71828
>>> x
2.71828
>>> x = 'two'
>>> x
'two'
```

The assignment statement itself doesn't return a value, so nothing is printed after each assignment statement (more precisely, Python's `None` value is returned, and the interpreter does not print anything when the typed expression evaluates to `None`). In Python 3.8, a new operator, `:=`, was introduced that

allows assignment in an expression that returns a value. For example, evaluating

```
(x := 2) ** 2 + (y := 3) ** 2
```

sets `x` to be 2 and `y` to be 3 and returns 13 as a value.

The type of a variable’s value can be determined by the `type()` function or tested using the `isinstance()` function.

Sequences

Arrays are different in Python than in other languages. The built-in data type that “looks like” an array is called a **list**. Python’s `list` type is a hybrid of the arrays and linked lists found in other languages. As with variables, the elements of Python lists are dynamically typed, so they do not all have to be of the same type. The maximum number of elements does not need to be declared when creating a `list`, and `lists` can grow and shrink at runtime. What’s quite different between Python `lists` and other linked list structures is that they can be indexed like arrays to retrieve any element at any position. There is no data type called `array` in the core of Python, but there is an `array` module that can be imported. The `array` module allows for construction of arrays of fixed-typed elements.

In this book, we make extensive use of the built-in `list` data type as if it were an array. This is for convenience of syntax, and because the underlying implementation of the `list` acts like arrays do in terms of indexed access to elements. Please note, however, that we do not use all of the features that the Python `list` type provides. The reason is that we want to show how fixed-type, fixed-length arrays behave in all computer languages. For new programmers, it’s better to use the simpler syntax that comes with the built-in `list` for constructing arrays while learning how to manipulate their contents in an algorithm.

Python’s built-in lists can be indexed using 0-relative indexes. For example:

```
>>> a = [1, 'a', False]
>>> a
[1, 'a', False]
>>> len(a)
```

```

3
>>> a[2]
False
>>> a[2] = True
>>> a
[1, 'a', True]
>>>

```

The three-element list in the example contains an integer, a string, and a Boolean value. The square brackets are used either to create a new list (as in the first assignment to `a`) or to enclose the index of an existing list (as in `a[2]`). The built-in `len()` function is used to determine the current size of its argument. Individual values in the list can be changed using the assignment statement. Strings can be treated like lists or arrays of characters, except that unlike lists, strings are immutable, meaning that after a string is created, the characters of the string cannot be changed. In Python 3, strings always contain Unicode characters, which means there can be multiple bytes per character.

```

>>> s = 'π = 3.14159'
>>> s
'π = 3.14159'
>>> len(s)
11
>>> π = 3.14159
>>> π
3.14159

```

In the preceding example, the string, `s`, contains the Greek letter π , which is counted as one character by the `len()` function, whereas the Unicode character takes two bytes of space. Unicode characters can also be used in variable names in Python 3 as shown by using π as a variable name.

Python treats all the data types that can be indexed, such as lists, arrays, and strings, as **sequence data types**. The sequence data types can be **sliced** to form new sequences. Slicing means creating a subsequence of the data, which is equivalent to getting a substring for strings. Slices are specified by a start and end index, separated by a colon (`:`) character. Every element from the start index up to, but not including, the end index is copied into the new sequence. The start index defaults to 0, the beginning of the sequence, and the end index defaults to the length of sequence. You can use negative numbers for both array and slice indexes. Negative indices count backwards from the end of the

sequence; -1 means the last element, -2 means the second to last element, and so on. Here are some examples with a string:

```
>>> digits = '0123456789'
>>> digits[3]
'3'
>>> digits[-1]
'9'
>>> digits[-2]
'8'
>>> digits[3:6]
'345'
>>> digits[:-2]
'01234567'
```

Sequence data types can be **concatenated**, **multiplied**, **searched**, and **enumerated**. These operations sometimes require more function calls in other languages, but Python provides simple syntax to perform them. For example:

```
>>> [1, 2, 3] + ['a', 'b']
[1, 2, 3, 'a', 'b']
>>> '011' * 7
'011011011011011011011'
>>> '011' * 0
''
>>> 3 in [1, 2, 3]
True
>>> 'elm' in ['Elm', 'Asp', 'Oak']
False
```

The preceding example shows two lists **concatenated** with the plus (+) operator to form a longer list. Multiplying a string by an integer produces that many copies of the string, concatenated together. The `in` operator is a Boolean test that **searches** for an element in a sequence. It uses the `==` equality test to determine whether the element matches. These operations work with all sequence data types. This compact syntax hides some of the complexity of stepping through each sequence element and doing some operation such as equality testing or copying the value over to a new sequence.

Looping and Iteration

Frequently we want to implement algorithms that process each of the elements of a sequence in order. For that, Python has several ways to **iterate** and **enumerate** sequences. For example:

```
>>> total = 0
>>> for x in [5, 4, 3, 2, 1]:
...     total += x
...
>>> total
15
```

The `for variable in sequence` syntax is the basic loop construct (or **iteration**) in Python. The nested expression is evaluated once for each value in the sequence with the variable bound to the value. There is no need to explicitly manipulate an index variable that points to the current element of the sequence; that's handled by the Python interpreter. One common mistake when trying to enter this expression in the interactive interpreter is to forget the empty line after the nested expression.

```
>>> total = 0
>>> for x in [5, 4, 3, 2, 1]:
...     total += x
...     total
File "<stdin>", line 3
    total
    ^
SyntaxError: invalid syntax
```

The reason this is so common is that the empty line is needed only for the interactive interpreter; the same Python expressions written in a file would not report this as an error. The interactive interpreter, however, waits for the empty line to signal the end of the `for` loop and begin evaluation of that full expression. When the interpreter finds a new expression starting at the same indent level as the `for` loop, it is dealing with two consecutive expressions and does not allow it. The interpreter expects to read one expression, evaluate it, and print the value, before starting to determine where the next expression begins and ends.

In some circumstances, having an explicit index variable is important. In those cases, there are a couple of convenient ways to perform the work. For example:

```

>>> height = [5, 4, 7, 2, 3]
>>> weightedsum = 0
>>> for i in range(len(height)):
...     weightedsum += i * height[i]
...
>>> weightedsum
36
>>> for i, h in enumerate(height):
...     weightedsum += i * h
...
>>> weightedsum
72

```

The example calculates a weighted sum where we multiply each value in the `height` list by the index of that value. The `range()` function can be thought of as a function that produces a list of integers starting at 0 and going up to, but not equal to, its argument. By passing `len(height)` as an argument, `range()` produces the list `[0, 1, 2, 3, 4]`. In the body of the first `for` loop, the `weightedsum` variable is incremented by the product of the index, `i`, and the value that `i` indexes in the `height` list. The second `for` loop repeats the same calculation using a slightly more concise form called **enumeration**. The `enumerate()` function can be thought of as taking a sequence as input and producing a sequence of pairs. The first element of each pair is an index, and the second is the corresponding value from its sequence argument. The second `for` loop has two variables separated by a comma, `i` and `h`, instead of just one in the previous loops. On each iteration of the `enumerate` loop, `i` is bound to the index and `h` is bound to the corresponding value from `height`. Python makes the common pattern of looping over a sequence very easy to write, both with or without an index variable.

The `range()` and `enumerate()` functions actually create *iterators*, which are complex data types that get called in each loop iteration to get the next value of the sequence. It doesn't actually produce a list in memory for the full sequence. We discuss how iterators can be used to represent very long sequences without taking up much memory in [Chapter 5](#), “[Linked Lists](#).”

Multivalued Assignment

The comma-separated list of variables can also be used in assignment statements to perform multiple assignments with a single equal sign (`=`)

operator. This makes the most sense when all the values being assigned are closely related, like coordinates of a vector. To illustrate:

```
>>> x, y, z = 3, 4, 5
>>> y
4
>>> (x, y, z) = [7, 8, 9]
>>> y
8
```

The sequences on both sides of the assignment operator must have the same length; otherwise, an error occurs. The sequences can be of different types. The second assignment here uses a **tuple** on the left and a list on the right side. Python's tuple data type is also a sequence data type very similar to a list, with the distinction that its elements cannot be modified. In this case, the tuple on the left must be a tuple of variable names. The comma is the operator that creates the tuple of the `x`, `y`, and `z` variables. Each of them is bound to the corresponding value in the list on the right. Looking back at the `enumerate` loop in the previous example, the iterations of the loop are performing the equivalent of

```
>>> i, h = (0, 5)
>>> weightedsum += i * h
>>> i, h = (1, 4)
>>> weightedsum += i * h
>>> i, h = (2, 7)
>>> weightedsum += i * h
>>> i, h = (3, 2)
>>> weightedsum += i * h
>>> i, h = (4, 3)
>>> weightedsum += i * h
```

Python's multivalued assignments can be used to swap or rotate values. That can be a surprise to programmers of other languages where an explicit temporary variable must be used to hold one or more of the values. For example:

```
>>> left, middle, right = 'Elm', 'Asp', 'Oak'
>>> left, middle, right
('Elm', 'Asp', 'Oak')
>>> left, middle, right = middle, right, left
```

```
>>> left, middle, right
('Asp', 'Oak', 'Elm')
```

The Python interpreter evaluates all the expressions on the right of the equal sign, puts the results in a tuple, and then makes the assignments to the variables on the left side from the tuple. The tuple holding the results is something like a hidden temporary variable. The parentheses surrounding the tuples on both sides of the equal sign operator are optional.

There is another kind of assignment statement that looks like the multivalued assignment but is different. You use it to assign the same value to multiple variables. For example:

```
>>> left = middle = right = 'Gum'
>>> left, middle, right
('Gum', 'Gum', 'Gum')
```

Several variables are all assigned the same value. The overall assignment statement, however, still evaluates as `None`.

Importing Modules

Functions can return multiple values in the form of tuples, too. You can decide whether to store all these values as a single tuple, or in several variables, one for each of the tuple's elements, just like the multivalued assignment does. A good example of this is splitting pathname components. The `os` module is in Python's standard library and provides many tools to operate on the underlying operating system where Python is running. To access a module, you import it. After it is imported, you refer to its contents via the module name, followed by a period, and then the name of one of its functions or other definitions. For example:

```
>>> import math
>>> math.pi
3.141592653589793
>>> import os
>>> os.path.splitext('myfile.ext')
('myfile', '.ext')
>>> filename, extension = os.path.splitext('myfile.ext')
>>> extension
'.ext'
```

The import statement creates a **namespace** for all the objects it defines. That's important to avoid conflicts between a variable or function that you define and the one in the module. In the preceding example, the value for `pi` is stored in the `math` namespace. If you have a separate definition, say an approximation like `pi = 3.1`, your program can refer to each one without confusion. The `os` module has submodules, one of which is the `path` module. Nested submodules create nested namespaces with corresponding names separated by periods. The `os.path.splitext()` function splits a filename at the last period and returns the two parts as a tuple. The preceding example shows binding that result to two variables, `filename` and `extension`, and then shows the value of the extension. Occasionally, you may call a function that returns multiple values without needing to retain all of them. Some Python programmers like to use the simplest variable name, a single underscore (`_`), as a throwaway variable. For instance, if only the filename extension was needed, you could write

```
_, extension = os.path.splitext('myfile.ext')
```

Functions and Subroutines

Of course, the core of all programming languages is the ability to define functions and subroutines. In Python, they are defined with the `def` statement. The parameters the function accepts are provided in a list of variables. Parameters can be mandatory or optional; the optional ones must have a default value to use when they are missing. The `def` statement has the name of the function, the parameter list in parentheses, a colon, followed by a nested expression that is the body of the routine. If it is a function, the body should contain one or more `return` statements for the value to be returned. Subroutines in other programming languages do not return values. In Python, all functions and subroutines return something, but if no explicit return value is provided in the body of a `def` statement, then the value returned is `None`.

Here's an implementation of a weighted sum function. It takes a sequence of weights and a sequence of values. If the `weights` sequence is shorter than that of the `values`, it uses the `missing` value as the default weight.

```
>>> def weightedsum(values, weights, missing=0):
...     sum = 0
...     for i, val in enumerate(values):
...         sum += val * (weights[i] if i < len(weights) else missing)
```

```

...     return sum
...
>>> weightedsum([4, 9, 16, 25], [2, 2])
26
>>> weightedsum([4, 9, 16, 25], [2, 2], 1)
67

```

The preceding example also illustrates Python's **conditional expression**, which has the following form:

```
expression if test_expression else expression
```

First the *test_expression* is evaluated. If it is true, then the leftmost *expression* is evaluated and returned. Otherwise, the rightmost *expression* is evaluated and returned.

We refer to the variables in the function's `def` statement as its **parameters**. The **arguments** are the values that the caller submits to the function. Most of the time, arguments are matched to their parameters by their position in the argument list. Keyword arguments, in contrast, can be in any order but must follow any positional arguments in the call. A function call can specify a parameter's value by preceding the value with the parameter's name and an equal sign (`=`). Function definitions can also treat a sequence of parameters as a list. These are marked in the function definition using an asterisk (`*`) before the parameter name. As an example, Python's `print` function accepts any number of objects to print along with multiple keyword parameters that control what separators it uses in printing and where and how it produces the output. Its definition looks like this:

```
def print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False):
```

Inside the body of the `print` subroutine, the `objects` parameter holds the list of things to print. The printed output is the string representation of each object value separated by a space character and terminated by a newline, by default. Normally, the output string is sent to the standard output stream and can be buffered. If, instead, you wanted to send the output to the standard error stream separated by tab characters, you could write

```

>>> import sys
>>> print(print, 'is a', type(print), file=sys.stderr, sep='\t')

```

```
<built-in function print>      is a <class  
'builtin_function_or_method'>
```

In this example, the call to `print` has three positional parameters that all get bound to the `objects` list. Note that the `file` and `sep` arguments are not in the same position as their corresponding parameters.

List Comprehensions

Applying a calculation or function on each element of a list to produce a new list is such a useful concept that Python has a special syntax for it. The concept is called a **list comprehension**, and it is written as a little loop inside the brackets used to create lists:

```
[expression for variable in sequence]
```

The `for variable in sequence` part is the same syntax as for procedural loops. The *expression* at the beginning is the same as the loop body (except that it can't contain multiple statements; only a single expression is allowed). The Python interpreter goes through the *sequence* getting each element and binding the *variable* to that element. It evaluates the *expression* in the context of the bound *variable* to compute a value. That computed value is put in the output sequence in the same position as the element from the input *sequence*. Here are two equivalent ways to produce a list of squares of the values in another list:

```
>>> values = [7, 11, 13, 17, 19]
>>> squares = []
>>> for val in values:
...     squares.append(val * val)
...
>>> squares
[49, 121, 169, 289, 361]
>>> [ x * x for x in values ]
[49, 121, 169, 289, 361]
```

The first few lines in the example show a simple loop that appends the squares to an initially empty list. The last line in the example collapses all those lines (other than defining the values) into a single expression. The whitespace just inside the square brackets is optional, but many programmers put it in to make it clearer that this is a list comprehension instead of a list created by evaluating a series of comma-separated expressions.

The list comprehension is a very compact syntax for describing the incredibly useful concept of **mapping**. It hides all the implementation details of indices and loop exit conditions while still making the essence of the operation very clear. Mapping is used everywhere in programming. In many applications the same operation needs to be applied over huge amounts of data and collected into a similar structure to the input.

Let's look at a couple more comprehensions to get the idea. To get the cube of all the integers between 10 and 20, you can write

```
>>> [ x ** 3 for x in range(10, 21) ]  
[1000, 1331, 1728, 2197, 2744, 3375, 4096, 4913, 5832, 6859, 8000]
```

To split hyphenated words, you could write

```
>>> [ w.split('-') for w in ['ape', 'ape-man', 'hand-me-down'] ]  
[['ape'], ['ape', 'man'], ['hand', 'me', 'down']]
```

The loop can also have a filter condition. Only those elements satisfying the condition are put in the output sequence. The filter expression goes at the end, that is,

```
[expression for variable in sequence if filter_expression]
```

The filter expression should evaluate to `True` for those sequence elements you wish to keep in the output. For example, let's get the cube of all the integers between 10 and 20 that are not multiples of 3:

```
>>> [ x ** 3 for x in range(10, 21) if x % 3 != 0 ]  
[1000, 1331, 2197, 2744, 4096, 4913, 6859, 8000]
```

And find all the characters in a string that are alphabetic:

```
>>> [ c for c in 'We, the People...' if c.isalpha() ]  
['W', 'e', 't', 'h', 'e', 'P', 'e', 'o', 'p', 'l', 'e']
```

These compact forms make it easy to describe the core operation while hiding many of the looping details needed to implement basic operations.

Exceptions

Python allows programs to define what happens when particular exception conditions occur. These exceptions aren't just for errors; Python uses exceptions to handle things like the end of iteration for loops, keyboard interrupts, and timer expiration, which are all expected events. Your programs can define new kinds of exceptions and ways to handle them if they occur. Exception handling can be defined within `try except` statements. The basic form is

```
try:
    <statements>
except:
    <statements>
```

The set of statements after the `try:` are executed from top to bottom, and if and only if an exception occurs during that execution, the statements after the `except:` are executed. If no exception occurs, then the next logical line after the `try except` statement is executed.

Programs can define different handlers for different kinds of exceptions. For example:

```
try:
    long_running_function()
except KeyboardInterrupt:
    print('Keyboard interrupted long running function')
except IndexError:
    print('Index out of range during long running function')
```

Each of the `except` clauses can specify a type (class) of exception. Exceptions of that type (or a subclass of the declared class) trigger the corresponding statements to be executed. When no exception class is specified in an `except` clause, as in the basic form example, any type of exception triggers the execution of the clause.

When a program doesn't specify an exception handler for the type of exception that occurs, Python's default exception handlers print out information about the exception and where it occurred. There are also optional `else` and `finally` clauses for `try except` statements, but we don't use them in this book.

You can specify exceptions in your programs by using the `raise` statement, which expects a single argument, an exception object, describing the condition. The statement

```
raise Exception('No more tickets')
```

raises a general exception with a short description. If the program has defined an exception handler for the general `Exception` class at the time this exception is raised, then it is used in place of Python's default handler. Otherwise, Python prints a stack trace and exits the program if you're not running in the interactive interpreter. We look more at exception objects in the next section.

Object-Oriented Programming

Object-oriented programming developed as a way to organize code for data structures. The data managed in the structure is kept in an **object**. The object is also called an **instance** of a **class of objects**. For example, in distributing tickets for an event, you would want an object to hold the list of phone numbers for people desiring tickets. The class of the object could be a queue that makes it easier to implement a first-come, first-served distribution system (as described in [Chapter 4](#), “[Stacks and Queues](#)”). Object classes define specific **methods** that implement operations on the object instances. For the ticketing system, there needs to be a method to add a new phone number for someone desiring tickets. There also needs to be methods to get the next phone number to call and to record how many tickets are assigned to each person (which might be stored in the same or different object). The methods are common to all objects in the class and operate on the data specific to each instance. If there were several events where tickets are being distributed, each event would need its own queue instance. The method for adding a phone number is common to each one of them and is **inherited** from the object class.

Python defines classes of objects with the `class` statement. These are organized in a hierarchy so that classes can inherit the definitions in other classes. The top of that hierarchy is Python's base class, `object`. The nested statements inside a `class` statement define the class; the `def` statements define methods and assignment statements define class attributes. The first parameter of each defined method should be `self`. The `self` parameter holds the object instance, allowing methods to call other instance methods and reference instance attributes. You reference an object instance's methods or attributes by

appending the method or attribute name to the variable holding the object, separated by a period as in `object.method()`.

To define the constructor for object instances, there should be a `def __init__()` statement inside the class definition. Like the other methods, `__init__()` should accept `self` as its first parameter. The `__init__()` method takes the empty instance and performs any needed initialization like creating instance variables and setting their values (and doesn't need to return `self` like constructors do in other languages). The `__init__()` method is unusual in that you will rarely see it called explicitly in a program (that is, in the form `variable.__init__()`). Python has several special methods and other constructs that all have names beginning and ending with double underscores. We point out a few of them as we use them in examples in the text.

The short example in [Listing 1-2](#) illustrates the basic elements of object-oriented programming in Python, along with some of its ability to handle fancy math concepts.

Listing 1-2 *Object-Oriented Program Example, `Object_Oriented_Client.py`*

```
class Power(object):
    """A class that computes a specific power of other numbers.
    In other words, it raises numbers by a constant exponent.
    """

    default_exponent = 2

    def __init__(self, exponent=default_exponent):
        self.exponent = exponent

    def of(self, x):
        return x ** self.exponent

class RealPower(Power): # A subclass of Power for real numbers

    def of(self, x):
        if isinstance(self.exponent, int) or x >= 0:
            return x ** self.exponent
        raise ValueError(
            'Fractional powers of negative numbers are imaginary')

print('Power:', Power)
print('Power.default_exponent:', Power.default_exponent)
```

```
square = Power()
root = Power(0.5)
print('square: ', square)
print('square.of(3) =', square.of(3))
print('root.of(3) =', root.of(3))
print('root.of(-3) =', root.of(-3))
real_root = RealPower(0.5)
print('real_root.of(3) =', real_root.of(3))
print('real_root.of(-3) =', real_root.of(-3))
print('Done.')
```

[Listing 1-2](#) shows a file with two class definitions, three object instances, and some print statements to show how the objects behave. The purpose of the first class, `Power`, is to make objects that can be used to raise numbers to various exponents. That is explained in the optional document string that follows the `class` definition. Note that this is not a Python comment, which would have to start with a pound symbol (`#`) on the left.

Each object instance is created with its own exponent, so you can create `Power` objects to perform different power functions. The `Power` class has one class attribute, `default_exponent`. This is used by the constructor for the class to define what exponent to use if none is provided when an instance is created.

The constructor for the `Power` class simply stores the desired exponent as an **instance attribute**. This is somewhat subtle because there are three distinct kinds of storage: **class attributes**, **instance attributes**, and **local variables** in the methods. The `default_exponent` attribute is defined by the assignment in the top level of the `class` statement, so it is a **class attribute** and is shared among all instances. The `exponent` parameter in the `__init__()` method of `Power` is a **local variable** available only during the evaluation of the constructor. It has a default value that is supplied from the class attribute.

When `__init__()` assigns a value to `self.exponent`, it creates an **instance attribute** for `self`. The instance attribute is unique to the object instance being created and is not shared with other objects of the class.

The `Power` class has one method called `of` that returns the result of raising a number to the exponent defined at instance creation. We use it after creating two instances, `square` and `root`, that can be called using `square.of(3)`, for example. To create the first instance, the program calls `Power(2)` and binds the result to the variable `square`. This behavior might be somewhat unexpected because there is no mention of `__init__()`. This is an example of Python's use of reserved names like `__init__()` to fill special roles such as object

constructors. When a class like `Power` is referenced using a function call syntax—followed by a parenthesized list of arguments—Python builds a new instance of the class and then invokes the `__init__()` method on it. The program in [Listing 1-2](#) makes two calls to `Power()` to make one object that produces squares and one that produces square roots.

Underscores in Python names mean that the item being defined is either special to the interpreter or to be treated as private, visible only in the block where it's defined. Python uses several special names like `__init__` for customizing the behavior of objects and programs. The special names start and end with double underscores and enable the use of tools like using the class name as constructor call.

In the example of [Listing 1-2](#), both the `default_exponent` class attribute and the `exponent` instance attribute are public because their names don't start with an underscore. If the name were changed to begin with an underscore, they are expected to be accessed only by the class and its methods and not to be accessed outside of the class or object. The restriction on access, however, is only a convention; the Python interpreter does not enforce the privacy of any attributes regardless of their name, nor does it provide other mechanisms to enforce privacy. Python does have a mechanism to mangle the names of class and object attributes that begin with a double underscore and end with at most one underscore. The name mangling is designed to make class-specific attributes that are not shared with subclasses, but they are still publicly accessible through the mangled names.

You can run the program by giving the filename as an argument to the Python interpreter program as follows (the colored text in *blue italics* is the text that you type):

```
$ python3 Object_Oriented_Client.py
Power: <class '__main__.Power'>
Power.default_exponent: 2
square: <__main__.Power object at 0x10715ad50>
square.of(3) = 9
root.of(3) = 1.7320508075688772
root.of(-3) = (1.0605752387249068e-16+1.7320508075688772j)
real_root.of(3) = 1.7320508075688772
Traceback (most recent call last):
  File "01_code/Object_Oriented_Client.py", line 32, in <module>
    print('real_root.of(-3) =', real_root.of(-3))
  File "01_code/Object_Oriented_Client.py", line 20, in of
```

```
'Fractional powers of negative numbers are imaginary')
ValueError: Fractional powers of negative numbers are imaginary
```

The transcript shows the output of the `print` statements. `Power` is a class, and `square` is an object instance of that class. The `square` instance is created by the call to `Power()` with no arguments provided, so the exponent defaults to 2. To create a similar way to compute square roots, the object created by `Power(0.5)` is assigned to `root`. The square of 3 printed by the program is the expected 9, and the square root of 3 is the expected 1.732.

The next `print` statement is for the square root of -3 , which may be a less expected result. It returns a **complex number** where `j` stands for the square root of -1 . While that may be interesting for some engineering applications, you might want a different behavior for other programs. The `RealPower` class is a subclass of the `Power` class that raises an exception when raising negative numbers to fractional powers.

In the `class` statement for `RealPower`, it is defined to inherit from the `Power` class. That means that `RealPower` will have the same `default_exponent` attribute and `__init__` constructor as `Power` does. In the class definition for `RealPower`, it replaces the `of` method with a new one that tests the values of the exponent and the numeric argument. If they fall in the category that produces imaginary numbers, it raises a `ValueError` exception.

The transcript shows how the Python interpreter handles exceptions. It prints a traceback showing the function and method calls that were being evaluated when the exception occurred. The traceback includes the line numbers and copies of the lines from the input file. After encountering the exception, the interpreter prints the traceback, quits, and does not evaluate the final `print` statement in the file.

Summary

- Data can be arranged in the computer in different ways and using various storage media. The data is organized and interpreted to represent something.
- Algorithms are the procedures used to manipulate data.
- By coupling good data organization with appropriate algorithms, data structures provide the fundamental building block of all programs.

- Examples of data structures are stacks, lists, queues, trees, and graphs.
- Data structures are often compared by how efficiently they perform common operations.
- A database is a collection of many similar records, each of which describes some entity.
- The records of a database are composed of fields, each of which has a name and a value.
- A key field is used to search for and sort records.
- Data structures that act like databases support four core operations: insertion, search, deletion, and traversal.
- Data structures are implemented as classes using object-oriented programming.
- Python has rich support for object-oriented programming with classes to implement data structures.

Questions

These questions are intended as a self-test for readers. The answers can be found in [Appendix C](#).

1. Data structures that allow programs to _____ a record, _____ for a record, _____ a record, and _____ all the records are considered to be databases in this book.
2. Data structures are
 - a. composed of names and values.
 - b. built with fields that have methods attached to them.
 - c. implemented with object classes.
 - d. records that don't change in a database.
3. How can you tell if a `def` statement in Python defines a function or class method?
4. What algorithms can help make searching more efficient?

5. What are constructors used for? What special name does Python use for them?
6. What are some of the reasons for choosing one data structure over another?
7. What is a key field used for?
8. Good data organization can help with the speed of an algorithm, but what other benefits does it have?
9. For what purpose was object-oriented programming developed?
10. Which of the following are data structures used in programming?
 - a. traceback
 - b. heap
 - c. list comprehension
 - d. hash table
 - e. recipe
 - f. slices
 - g. binary tree

Experiments

Try the following experiments:

- 1-A Write a Python list comprehension that returns the individual characters of a string that are not whitespace characters. Apply it to the string `"4 and 20 blackbirds.\n"`
- 1-B Take a deck of playing cards, pull out the 13 spade cards, set aside the rest, and shuffle the spade cards. Devise an algorithm to sort them by number under the constraints:
 - a. All the cards must be held in one hand. This is the “first” hand.
 - b. Initially, the shuffled cards are all stacked with faces in one direction so that only one card is visible.
 - c. Initially, all the cards are held between the thumb and forefinger of the first hand.

- d. The visible card in the stack can be pulled out using the other hand and placed in between any of the fingers of the first hand. It can only be placed at the front or the back of the cards in the stack of cards between those fingers.
- e. The other hand can hold one card at a time and must place it somewhere in the first hand before picking out another visible card from one of the stacks.
- f. The algorithm is done when all the cards are in sorted order in one stack in the hand.

Compare the efficiency of your algorithm with that of classmates or friends.

2. Arrays

Arrays are the most commonly used data structure for many reasons. They are straightforward to understand and match closely the underlying computer hardware. Almost all CPUs make it very fast to access data at known offsets from a base address. Almost every programming language supports them as part of the core data structures. We study them first for their simplicity and because many of the more complex data structures are built using them.

In This Chapter

- [The Array Visualization Tool](#)
- [Using Python Lists to Implement the Array Class](#)
- [The Ordered Array Visualization Tool](#)
- [Binary Search](#)
- [Python Code for an Ordered Array Class](#)
- [Logarithms](#)
- [Storing Objects](#)
- [Big O Notation](#)
- [Why Not Use Arrays for Everything?](#)

First, we look at the basics of how data is inserted, searched, and deleted from arrays. Then, we look at how we can improve it by examining a special kind of array, the ordered array, in which the data is stored in ascending (or descending) key order. This arrangement makes possible a fast way of searching for a data item: the binary search.

To improve a data structure's performance requires a way of measuring performance beyond just running it on sample data. Looking at examples of how it handles particular kinds of data makes it easier to understand the operations. We also take the first step to generalize the performance measure by looking at linear and binary searches, and introducing Big O notation, the most widely used measure of algorithm efficiency.

Suppose you're coaching kids-league soccer, and you want to keep track of which players are present at the practice field. What you need is an attendance-monitoring program for your computer—a program that maintains a database of the players who have shown up for practice. You can use a simple data structure to hold this data. There are several actions you would like to be able to perform:

- Insert a player into the data structure when the player arrives at the field.
- Check to see whether a particular player is present, by searching for the player's number in the structure.
- Delete a player from the data structure when that player leaves.
- List all the players present.

These four operations—insertion, searching, deletion, and enumeration (traversal)—are the fundamental ones in most of the data storage structures described in this book.

The Array Visualization Tool

We often begin the discussion of a particular data structure by demonstrating it with a visualization tool—a program that animates the operations. This approach gives you a feeling for what the structure and its algorithms do, before we launch into a detailed explanation and demonstrate sample code. The visualization tool called Array shows how an array can be used to implement insertion, searching, and deletion.

Now start up the Array Visualization tool, as described in [Appendix A](#), “[Running the Visualizations](#).” There are several ways to do this, as described in the appendix. You can start it up separately or in conjunction with all the visualizations. If you've downloaded Python and the source code to your computer, you can launch it from the command line using

python3 Array.py

Figure 2-1 shows the initial array with 10 elements, 9 of which have data items in them. You can think of these items as representing your players. Imagine that each player has been issued a team shirt with the player's number on the back. That's really helpful because you have just met most of these people and haven't learned all their names yet. To make things visually interesting, the shirts come in a variety of colors. You can see each player's number and shirt color in the array. The height of the colored rectangle is proportional to the number.

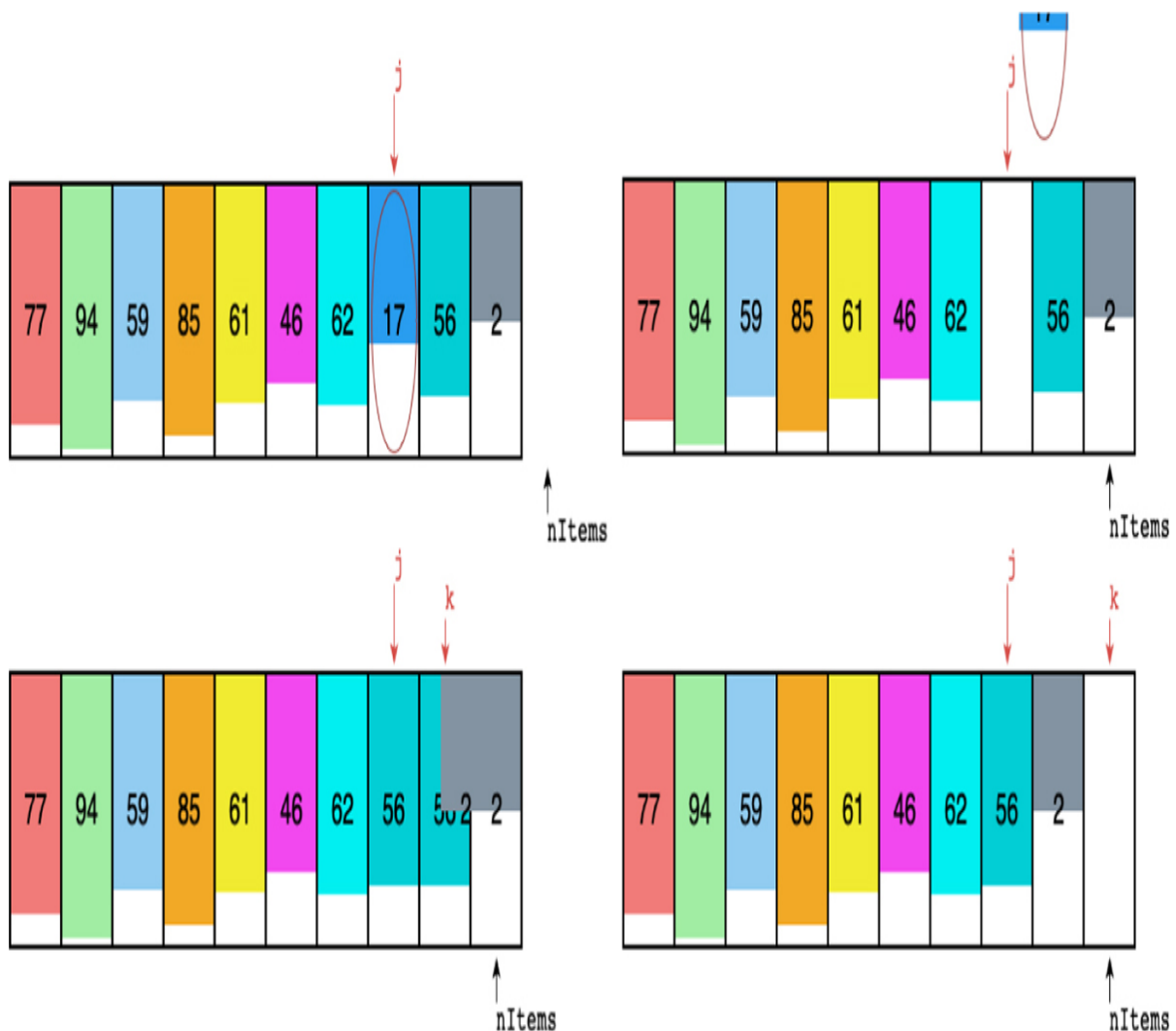


Figure 2-1 *The Array Visualization tool*

This visualization demonstrates the four fundamental operations mentioned earlier:

- The Insert button inserts a new data item.
- The Search button searches for specified data item.
- The Delete button deletes a specified data item.
- The Traverse button lists all the items in the array.

The buttons for the first three of those operations are on the left, grayed out and disabled. The reason is that you haven't entered a number in the small box to their right. The hint below the box suggests that when there's a number to search, insert, or delete, the buttons will be enabled.

The Traverse button is on the right and enabled. That's because it doesn't require an argument to start traversing. We explore that and the other buttons on the right shortly.

On the left, there is also a button labeled New. It's used to create a new array of a given size. The size is taken from the text entry box like the other arguments. (The initial hint shown in [Figure 2-1](#) also mentioned that you can enter the number of cells here.) Arrays must be created with a known size because they place the contents in adjacent memory cells, and that memory must be allocated for exclusive use by the array. We look at each of these operations in turn.

Searching

Imagine that you just arrived at the playing field to start coaching, your assistant hands you the computer that's tracking attendance, and a player's parent asks if the goalies can start their special drills. You know that players 2, 5, and 17 are the ones who play as goalies, but are they all here? Although answering this question is trivial for a real coach with paper and pencil, let's look at the details needed to do it with the computer.

You want to search to see whether all the players are present. The array operations let you search for one item at a time. In the visualization tool, you can select the text entry box near the Search button, the hint disappears, and you enter the number 2. The button becomes enabled, and you can select it to

start the search. The tool animates the search process, which starts off looking like [Figure 2-2](#).



Figure 2-2 *Starting a search for player 2*

The visualization tool now shows several new things. A box in the lower right shows a program being executed. Next to the array drawing, an arrow labeled with a `j` points to the first cell. The `j` advances to each cell in turn, checking to see whether it holds the number 2. When it reaches where the `nItems` arrow points on the right, all the cells have been searched. The `j` arrow disappears, and a message appears at the bottom: `Value 2 not found`.

This process mimics what humans would do—scan the list, perhaps with a finger dragged along the numbers, confirming whether any of them match the number being sought. The visualization tool shows how the computer represents those same activities. The tool allows you to pause the animation of the process if you want to look at the details. The three buttons at the bottom right of the operations area,   , control the animation. The leftmost one plays or pauses the animation. The middle button plays until the next code step, and the square shape on the right stops the operation. At the bottom, you can slide the animation speed control left or right to slow down or speed up the animation.

Try searching for the value 17 (or some other number present in the array). The visualization tool starts with the `j` arrow pointing at the leftmost array cell. After checking the value there, it moves to the next cell to the right. That process repeats seven times in the array shown in [Figure 2-2](#), and then the value is circled. After a few more changes in the code box, the message `Value 17 Found` appears at the bottom (we return to the code in a moment).

What's important to notice here is that the search went through seven steps before finding the value. When you searched for 2, it went through all nine items before stopping. Let's call the number of items `N` (which is just a shorter version of the `nItems` shown in the visualization). When you search for an item that is in the array, it could take 1 to `N` steps before finding it. If there's an equal chance of being asked to search for each item, the average number of search steps is $(1 + 2 + 3 + \dots + N-1 + N) / N$ which works out to $(N + 1) / 2$. Unsuccessful searches take `N` steps every time.

There's another simple but still important thing to notice. What if there were duplicate values in the array? That's not supposed to happen on the team, of course. Every player should have a distinct number. If one player lent one of their shirts to a teammate who forgot theirs, however, it would not be surprising to see the same number twice.

The search strategy must know whether to expect multiple copies of a number to exist. The visualization tool stops when it finds the first matching number. If multiple copies are allowed, *and* it's important to find all of them, then the search couldn't stop after finding the first one. It would have to go through all `N` items and identify how many matched. In that case, both successful and unsuccessful searches would take `N` steps.

Insertion

We didn't find player 2 when asked before, but now that player has just arrived. You need to record that player 2 is at practice, so it's time to insert them in the array. Type 2 in the text entry box and select Insert. A new colored rectangle with 2 in it appears at the bottom and moves into position at the empty cell indicated by the `nItems` pointer. When it's in position, the `nItems` pointer moves to the right by one. It may now point beyond the last cell of the array.

The animation of the arrival of the new item takes a little time, but in terms of what the computer has to do, only two steps were needed: writing the new value in the array at the position indicated by `nItems` and then incrementing `nItems` by 1. It doesn't matter if there were two, three, or a hundred items already in the array; inserting the value always takes two steps. That makes the insertion operation quite different from the search operation and almost always faster.

More precisely, the number of steps for insertion doesn't depend on how many items are in the array as long as it is not full. If all the cells are filled, putting a value outside of the array is an error. The visualization tool won't let that happen and will produce an error message if you try. (In the history of programming, however, quite a few programmers did not put that check in in the code, leading to buffer overflows and security problems.)

The visualization tool lets you insert duplicate values (if there are available cells). It's up to you to avoid them, possibly by using the Search operation, if you don't want to allow it.

Deletion

Player 17 has to leave (he wants to start on the homework assignment due tomorrow). To delete an item in the array, you must first find it. After you type in the number of the item to be deleted, a process like the search operation begins in the visualization tool. A `j` arrow appears starting at the leftmost cell and steps to the right as it checks values in the cells. When it finds and circles the value as shown in the top left of [Figure 2-3](#), however, it does something different.



Figure 2-3 *Deleting an item*

The visualization tool starts by reducing `nItems` by 1, moving it to point at the last item in the array. That behavior might seem odd at first, but the reason will become clear when we look at the code. The next thing it does is move the deleted value out of the array, as shown in the top right of [Figure 2-3](#). That doesn't actually happen in the computer, but the empty space helps distinguish that cell visually during what happens next.

A new arrow labeled k appears pointing to the same cell as j . Each of the items to the right of the deleted item is copied into the cell to its left, and k is advanced. So, for example, item 56 is copied into cell j , and then item 2 is copied into where 56 was, as shown in the bottom left of the figure. When they are all moved, the `nItems` arrow (and k) points at the empty cell just after the last filled cell, as shown at the bottom right. That makes the array ready to accept the next item to insert in just two steps. (The visualization tool clears the last cell after copying its contents to the cell to its left. This behavior is not strictly necessary but helps the visualization.)

Implicit in the deletion algorithm is the assumption that holes are not allowed in the array. A **hole** is one or more empty cells that have filled cells above them (at higher index numbers). If holes are allowed, the algorithms for all the operations become more complicated because they must check to see whether a cell is empty before doing something with its contents. Also, the algorithms become less efficient because they waste time looking at unoccupied cells. For these reasons, occupied cells must be arranged contiguously: no holes allowed.

Try deleting another item while carefully watching the changes to the different arrows. You can slow down and pause the animation to see the details.

How many steps does each deletion take? Well, the j arrow had to move over a certain number of times to find the item being deleted, let's call that J . Then you had to shift the items to the right of j . There were $N - J$ of those items. In total there were $J + N - J$ steps, or simply N steps. The steps were different in character: checking values versus copying values (we consider the difference between making value comparisons and shifting items in memory later).

Traversal

Arrays are simple to traverse. The data is already in a linear order as specified by the index to the array elements. The index is set to 0, and if it's less than the current number of items in the array, then the array item at index 0 is processed. In the Array visualization tool, the item is copied to an output box, which is similar to printing it. The index is incremented until it equals the current number of items in the array, at which point the traversal is complete. Each item with an index less than `nItems` is processed exactly once. It's very easy to traverse the array in reverse order by decrementing the index too.

The Duplicates Issue

When you design a data storage structure, you need to decide whether items with duplicate keys will be allowed. If you're working with a personnel file and the key is an employee number, duplicates don't make much sense; there's no point in assigning the same number to two employees. On the other hand, a list of contacts might have several entries of people who have the same family name. Two entries might even have the same given and family names.

Assuming the names are the key for looking up the contact, duplicate keys should be allowed in a simple contacts list. Another example would be a data structure designed to keep track of the food items in a pantry. There are likely to be several identical items, such as cans of beans or bottles of milk. The key could be the name of the item or the label code on its package. In this context, it's likely the program will not only want to search for the presence for an item but also count how many identical items are in the store.

If you're writing a data storage program in which duplicates are not allowed, you may need to guard against human error during an insertion by checking all the data items in the array to ensure that not one of them already has the same key value as the item being inserted. This check reduces the efficiency, however, by increasing the number of steps required for an insertion from one to N . For this reason, the visualization tool does not perform this check.

Searching with Duplicates

Allowing duplicates complicates the search algorithm, as we noted. Even if the search finds a match, it must continue looking for possible additional matches until the last occupied cell. At least, this is one approach; you could also stop after the first match and perform subsequent searches after that. How you proceed depends on whether the question is "Find me everyone with the family name of Smith," "Find me someone with the family name of Smith," or the similar question "Find how many entries have the family name Smith."

Finding all items matching a search key is an **exhaustive search**. Exhaustive searches require N steps because the algorithm must go all the way to the last occupied cell, regardless of what is being sought.

Insertion with Duplicates

Insertion is the same with duplicates allowed as when they're not: a single step inserts the new item. Remember, however, that if duplicates are prohibited, and

there's a possibility the user will attempt to input the same key twice, the algorithm must check every existing item before doing an insertion.

Deletion with Duplicates

Deletion may be more complicated when duplicates are allowed, depending on exactly how “deletion” is defined. If it means to delete only the first item with a specified value, then, on the average, only $N/2$ comparisons and $N/2$ moves are necessary. This is the same as when no duplicates are allowed. This would be the desired way to handle deleting an item such as a can of beans from a kitchen pantry when it gets used. Any items with duplicate keys remain in the pantry.

If, however, deletion means to delete *every* item with a specified key value, the same operation may require multiple deletions. Such an operation requires checking N cells and (probably) moving more than $N/2$ cells. The average depends on how the duplicates are distributed throughout the array.

Traversal with Duplicates

Traversal means processing each of the stored items exactly once. If there are duplicates, then each duplicate item is processed once. That means that the algorithm doesn't change if duplicates are present. Processing all the *unique* keys in the data store exactly once is a different operation.

Table 2-1 shows the average number of comparisons and moves for the four operations, first where no duplicates are allowed and then where they are allowed. N is the number of items in the array. Inserting a new item counts as one move.

Table 2-1 *Duplicates OK Versus No Duplicates*

	No Duplicates	Duplicates OK
<i>Search</i>	$N/2$ comparisons	N comparisons
<i>Insertion</i>	No comparisons, one move	No comparisons, one move
<i>Deletion</i>	$N/2$ comparisons, $N/2$ moves	N comparisons, more than $N/2$ moves
<i>Traversal</i>	N processing steps	N processing steps

The difference between N and $N/2$ is not usually considered very significant, except when you're fine-tuning a program. Of more importance, as we discuss toward the end of this chapter, is whether an operation takes one step, N steps, or N^2 steps, which would be the case if you wanted to enumerate all the pairs of items stored in a list. When there are only a handful items, the differences are small, but as N gets bigger, the differences can become huge.

Not Too Swift

One of the significant things to notice when you're using the Array Visualization tool is the slow and methodical nature of the algorithms. Apart from insertion, the algorithms involve stepping through some or all of the cells in the array performing comparisons, moves, or other operations. Different data structures offer much faster but slightly more complex algorithms. We examine one, the search on an ordered array, later in this chapter, and others throughout this book.

Deleting the Rightmost Item

Deletion is the slowest of the four core operations in an array. If you don't care what item is deleted, however, it's easy to delete the last (rightmost) item in the array. All that task requires is reducing the count of the number of items, `nItems`, by one. It doesn't matter whether duplicates are allowed or not; deleting the last item doesn't affect whether duplicates are present. The Array Visualization tool provides a Delete Rightmost operation to see the effect of this fast—one "move" or assignment no comparison—operation.

Using Python Lists to Implement the Array Class

The preceding section showed the primary algorithms used for arrays. Now let's look at how to write a Python class called `Array` that implements the array abstraction and its algorithms. But first we want to cover a few of the fundamentals of arrays in Python.

As mentioned in [Chapter 1](#), "Overview," Python has a built-in data structure called `list` that has many of the characteristics of arrays in other languages. In the first few chapters of this book, we stick with the simple, built-in Python constructs for lists and use them as if they were arrays, while avoiding the use of more advanced features of Python lists that might obscure the details of

what's really happening in the code. In [Chapter 5](#), “[Linked Lists](#),” we introduce linked lists and describe the differences. For those programmers who started with Python lists, it may seem odd to initialize the size of the `Array`'s list at the beginning, but this is a necessary step for true arrays in all programming languages. The memory to hold all the array elements must be allocated at the beginning so that the sequence of elements can be stored in a contiguous range of memory and any array element can be accessed in any order.

Creating an Array

As we noted in [Chapter 1](#), Python lists are constructed by enclosing either a list of values or by using a list comprehension (loop) in square brackets. The list comprehension really builds a new `list` based on an existing list or sequence. To allocate lists with large numbers of values, you use either an iterator like `range()` inside a list comprehension or the multiplication operator. Here are some examples:

```
integerArray = [1, 1, 2, 3, 5]    # A list of 5 integers
charArray = ['a' for j in range(1000)] # 1,000 letter 'a' characters
booleArray = [False] * 32768      # 32,768 binary False values
```

Each of these assignment statements creates a list with specific initial values. Python is dynamically typed, and the items of a list do not all have to be of the same type. This is one of the core differences between Python lists and the arrays in statically typed languages, where all items must be of the same type. Knowing the type of the items means that the amount of memory needed to represent each one is known, and the memory for the entire array can be allocated. In the case of `charArray` in the preceding example, Python runs a small loop 1,000 times to create the list. Although all three sample lists start with items of the same type, they could later be changed to hold values of any type, and their names would no longer be accurate descriptions of their contents.

Data structures are typically created empty, and then later insertions, updates, and deletions determine the exact contents. Primitive arrays are allocated with a given maximum size, but their contents could be anything initially. When you're using an array, some other variables must track which of the array elements have been initialized properly. This is typically managed by an integer that stores the current number of initialized elements.

In this book, we refer to the storage location in the array as an **element** or as a **cell**, and the value that is stored inside it as an **item**. In Python, you could write the initialization of an array like this:

```
maxSize = 10000
myArray = [None] * maxSize
myArraySize = 0
```

This code allocates a list of 10,000 elements each initialized to the special `None` value. The `myArraySize` variable is intended to hold the current number of inserted items, which is 0 at first. You might think that Python's built-in `len()` function would be useful to determine the current size, but this is where the implementation of an array using a Python list breaks down. The `len()` function returns the allocated size, not how many values have been inserted in `myArray`. In other words, `len(myArray)` would always be 10,000 (if you only change individual element values). We will track the number of items in our data structures using other variables such as `nItems` or `myArraySize`, in the same manner that must be done with other programming languages.

Accessing List Elements

Elements of a list are accessed using an **integer index** in square brackets. This is similar to how other languages work:

```
temp = myArray[3]    # get contents of fourth element of array
myArray[7] = 66      # insert 66 into the eighth cell
```

Remember that in Python—as in Java, C, and C++—the first element is numbered 0, so the indices in an array of 10 elements run from 0 to 9. What's different in Python is that you can use negative indices to specify the count from the end of the list. You can think of that as if the current size of the array were added to the negative index. In the previous example, you could get the last item by writing `myArray[maxSize - 1]` or, more simply, `myArray[-1]`.

Python also supports **slicing** of lists, but that seemingly simple operation hides many details that are important to the understanding of an `Array` class's behavior. As a result, we don't use slices in showing how to implement an `Array` class using lists.

Initialization

All the methods we've explored for creating lists in Python involve specifying the initial value for the elements. In other languages, it's easy to create or **allocate** an array without specifying any initial value. As long as the array elements have a known size and there is a known quantity of them, the memory needed to hold the whole array can be allocated. Because computers and their operating systems reuse memory that was released by other programs, the element values in a newly allocated array could be anything. Programmers must be careful not to write programs that use the array values before setting them to some desired value because the uninitialized values can cause errors and other unwanted behavior. Initializing Python list values to `None` or some other known constant avoids that problem.

An Array Class Example

Let's look at some sample programs that show how a list can be used. We start with a basic, object-oriented implementation of an `Array` class that uses a Python list as its underlying storage. As we experiment with it, we will make changes to improve its performance and add features. [Listing 2-1](#) shows the class definition, called `Array`. It's stored in a file called `BadArray.py`.

Listing 2-1 *The `BadArray.py` Module*

```
# Implement an Array data structure as a simplified type of list.

class Array(object):

    def __init__(self, initialSize):      # Constructor
        self.__a = [None] * initialSize # The array stored as a list
        self.nItems = 0                  # No items in array initially

    def insert(self, item):               # Insert item at end
        self.__a[self.nItems] = item     # Item goes at current end
        self.nItems += 1                  # Increment number of items

    def search(self, item):
        for j in range(self.nItems):     # Search among current
            if self.__a[j] == item:      # If found,
                return self.__a[j]       # then return item

        return None                       # Not found -> None
```

```

def delete(self, item):          # Delete first occurrence
    for j in range(self.nItems): # of an item
    if self.__a[j] == item:      # Found item
        for k in range(j, self.nItems): # Move items from
            self.__a[k] = self.__a[k+1] # right over 1
        self.nItems -= 1          # One fewer in array now
        return True              # Return success flag

    return False                # Made it here, so couldn't find the item

def traverse(self, function=print): # Traverse all items
    for j in range(self.nItems):    # and apply a function
        function(self.__a[j])

```

The `Array` class has a constructor that initializes a fixed length list to hold the array of items. The array items are stored in a private instance attribute, `__a`, and the number of items stored in the array is kept in the public instance attribute, `nItems`. The four methods define the four core operations.

Before we look at the implementation details, let's use a program to test each operation. A separate file, `BadArrayClient.py`, shown in [Listing 2-2](#) uses the `Array` class in the `BadArray.py` module. This program imports the class definition, creates an `Array` called `arr` with a `maxSize` of 10, inserts 10 data items (integers, strings, and floating-point numbers) in it, displays the contents by traversing the `Array`, searches for a couple of items in it, tries to remove the items with values 0 and 17, and then displays the remaining items.

Listing 2-2 *The `BadArrayClient.py` Program*

```

import BadArray
maxSize = 10          # Max size of the Array
arr = BadArray.Array(maxSize) # Create an Array object

arr.insert(77)        # Insert 10 items
arr.insert(99)
arr.insert("foo")
arr.insert("bar")
arr.insert(44)
arr.insert(55)
arr.insert(12.34)
arr.insert(0)
arr.insert("baz")
arr.insert(-17)

```